CloudGallery

A Photo Gallery Website Hosted on Amazon S3

Mohas Cloud Computing Training – Project 1

AWS Free Tier Hands-On Project (S3, IAM, Security & Monitoring)

Name: Nigatie Sahlie

S3 Bucket Name: `cloud-gallery-nigatie`

AWS Region: US East (N. Virginia) – us-east-1

Submitted to: **Mr. Anteneh**

Due Date: July 7, 2026

July 7, 2026

Contents

Project Overview

This report documents the completion of **CloudGallery**, a static photo gallery website built and hosted entirely on Amazon S3. The project demonstrates practical skills across five core areas of AWS cloud computing: S3 storage and static website hosting, IAM user creation with scoped permissions, S3 security controls (bucket policies, versioning), lifecycle management, and monitoring. The bucket used for this project is `cloud-gallery-nigatie`, created in the `us-east-1` region.

Concept	How It Was Covered
S3	Created bucket, uploaded files, enabled static hosting
IAM	Created `s3-uploader` user with S3 permissions
Security	Configured bucket policy and enabled versioning
Monitoring	Attempted CloudWatch (see Note in Part 4.4)
Networking	Verified S3 website endpoint and bucket URL
Linux	Verified all resources using AWS CLI on Ubuntu

Important note on project completion: All tasks through **Part 4.3 (Lifecycle Rule)** were completed successfully and are fully documented below with screenshots. After completing the lifecycle rule step, my AWS Free Tier account became inaccessible (it says account suspended) before I could complete **Task 4.4 (CloudWatch Monitoring)** and **Part 5 (Troubleshooting & Clean Up)**. This is explained in detail in the relevant sections, along with the conceptual answers that would otherwise have been demonstrated with screenshots.

1 Create S3 Bucket

1.1 Task 1.1: Create Bucket via Console

A new general-purpose S3 bucket named `cloud-gallery-nigatie` was created in the `us-east-1` (N. Virginia) region with ACLs disabled and "Block all public access" unchecked, since the bucket was intended to serve public static website content.

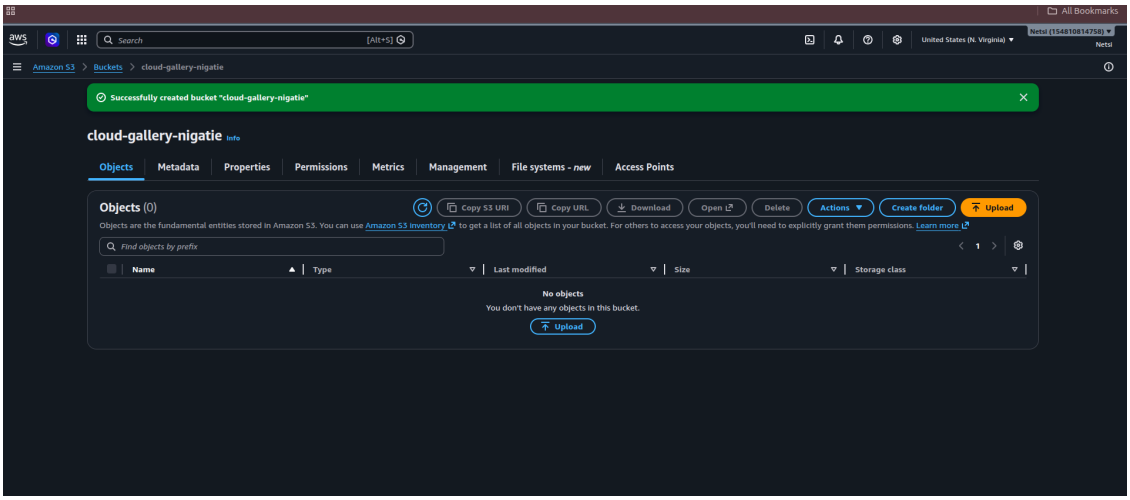

Figure 1: Screenshot 1 – S3 bucket `cloud-gallery-nigatie` successfully created.

The bucket was then verified using the AWS CLI:

```
$ aws s3 ls
2026-07-04 22:09:46 cloud-gallery-nigatie
```

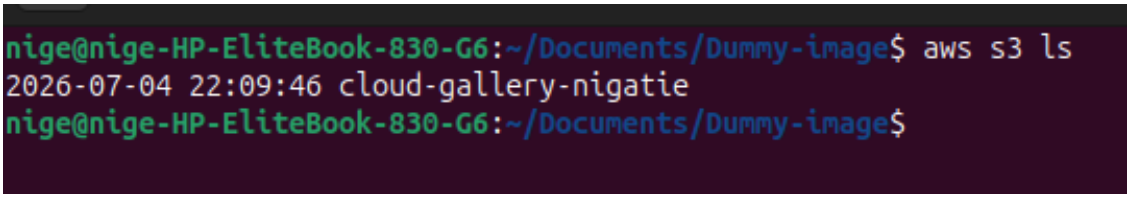

Figure 2: CLI verification confirming the bucket exists in the account.

1.2 Task 1.2: Upload Images to S3

Three dummy image files (`dummy.png`, `dummy1.png`, `dummy2.png`) were generated and uploaded to the bucket through the S3 console.

The upload was verified independently from the terminal using the AWS CLI:

```
$ aws s3 ls s3://cloud-gallery-nigatie
2026-07-04 23:14:21   185 dummy.png
2026-07-04 23:14:21   185 dummy1.png
2026-07-04 23:14:20   185 dummy2.png
```

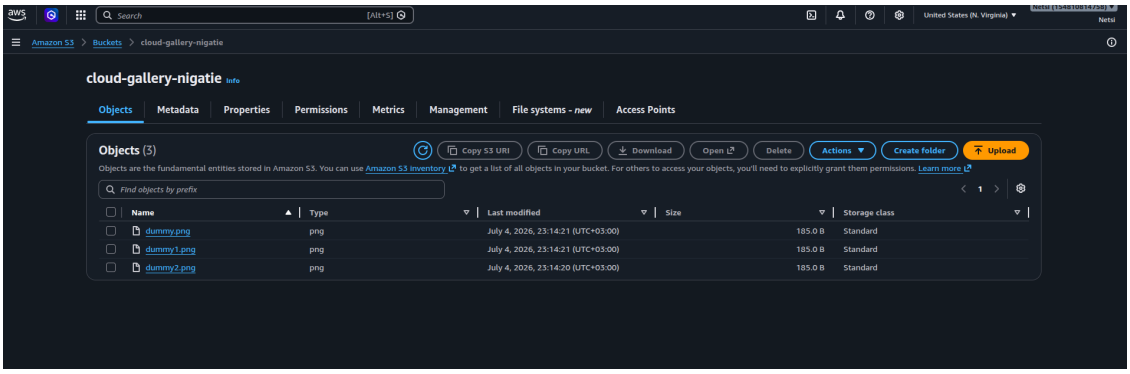

Figure 3: Screenshot 2 – Bucket contents showing the three uploaded image files.

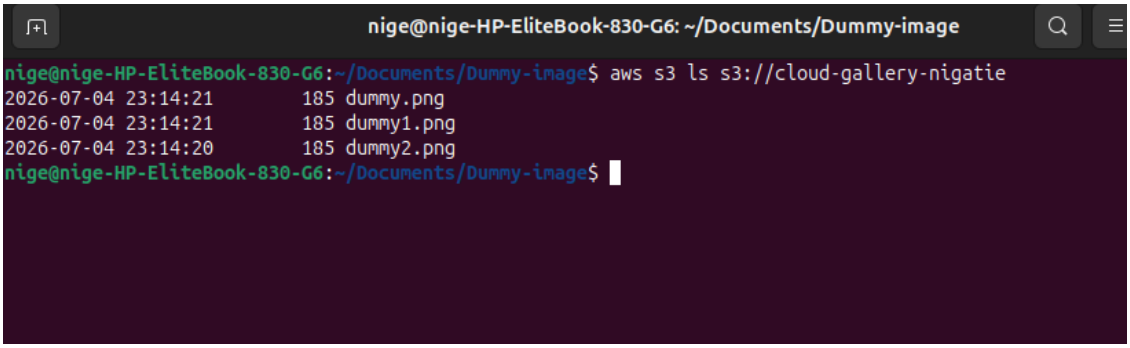

Figure 4: CLI verification of uploaded objects in the bucket.

2 Enable Static Website Hosting

2.1 Task 2.1: Configure Static Hosting

Static website hosting was enabled on the `cloud-gallery-nigatie` bucket from the **Properties** tab, with `index.html` set as the index document. The resulting bucket website endpoint was:

http://cloud-gallery-nigatie.s3-website-us-east-1.amazonaws.com

2.2 Task 2.2 & 2.3: Create Website Files and Test the Website

The provided `index.html` gallery page (styled with a purple gradient background, card-based photo layout, and a live "last updated" timestamp) was uploaded to the bucket root. Visiting the bucket website endpoint in a browser rendered the CloudGallery page correctly, with all three photo cards displaying their fallback SVG placeholders and the "S3 Hosted" badges.

3 Configure IAM for S3 Access

3.1 Task 3.1: Create IAM User

An IAM user named `s3-uploader` was created with the `AmazonS3FullAccess` managed policy attached directly, rather than using the AWS account root user for programmatic S3 access.

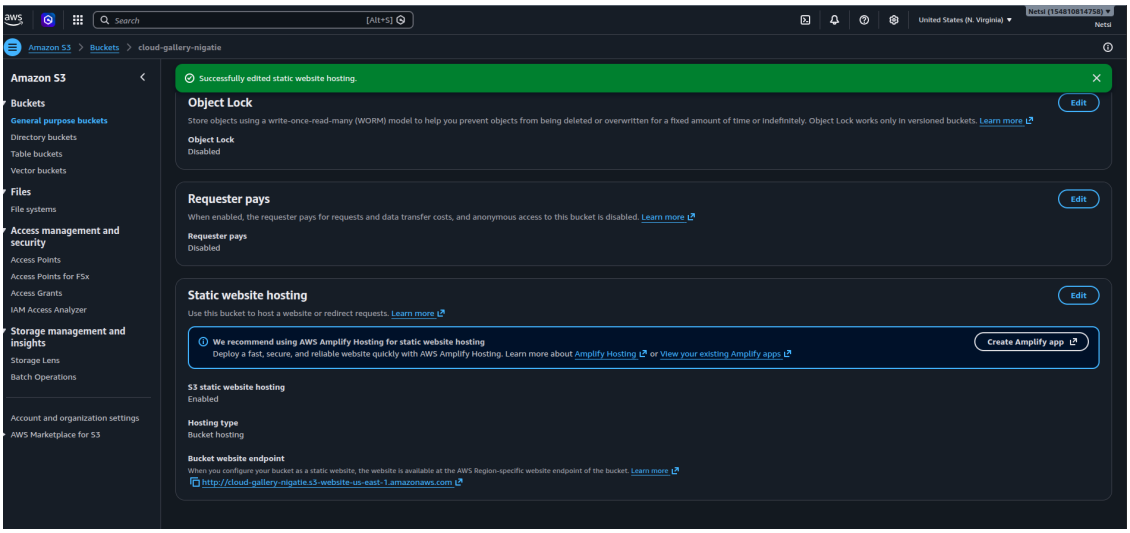

Figure 5: Screenshot 3 – Static website hosting enabled with the generated bucket website endpoint.

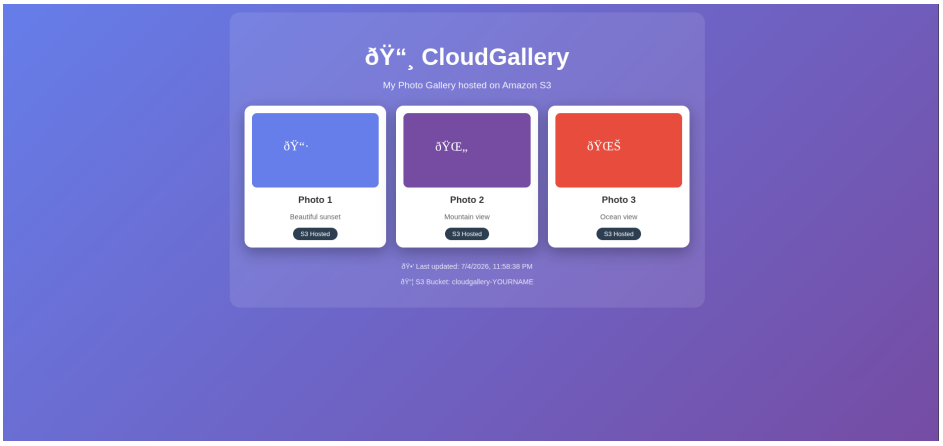

Figure 6: Screenshots 4 & 5 – Live CloudGallery website successfully loading from the S3 static website endpoint.

3.2 Task 3.2: Generate Access Keys

An access key of type "Command Line Interface (CLI)" was generated for the `s3-uploader` user. The secret access key was copied at creation time, since AWS only displays it once.

3.3 Task 3.3: Configure AWS CLI

The AWS CLI on the local Ubuntu machine (HP EliteBook 830 G6) was reconfigured with the new user's credentials using `aws configure`, targeting the `us-east-1` region. Running `aws s3 ls` confirmed that the `s3-uploader` credentials had the correct permissions to list the bucket.

```
$ aws s3 ls
2026-07-04 22:09:46 cloud-gallery-nigatie
```

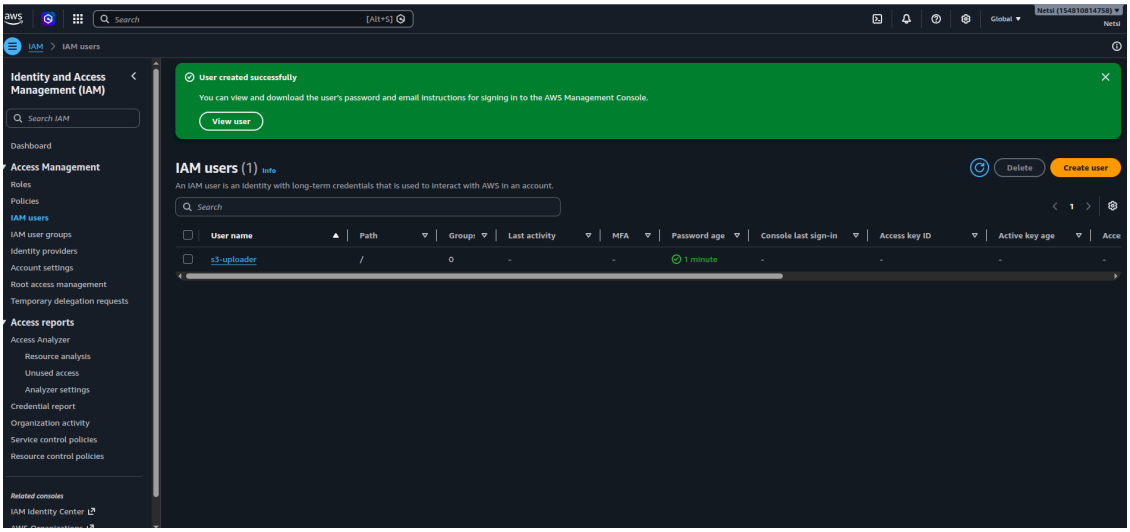

Figure 7: Screenshot 6 – IAM user `s3-uploader` created successfully.

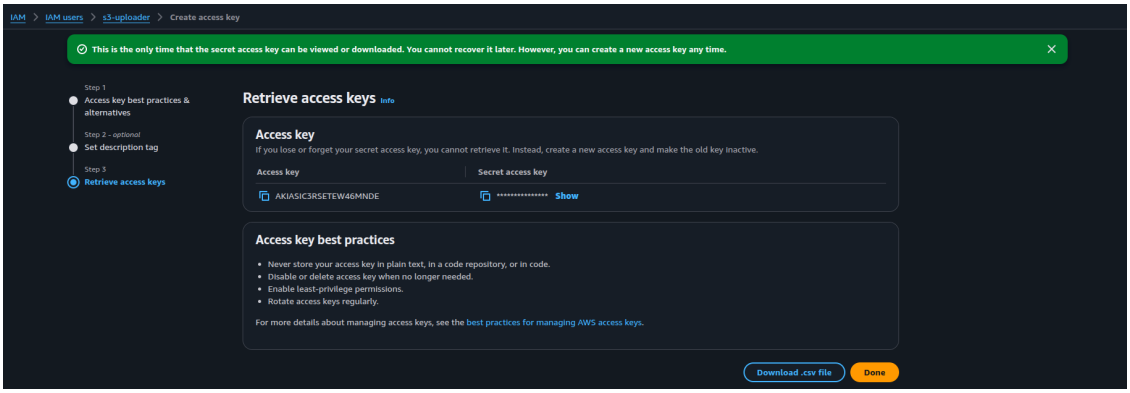

Figure 8: Screenshot 7 – Access key generated for `s3-uploader` (secret value redacted/hidden by default).

4 S3 Security & Monitoring

4.1 Task 4.1: S3 Bucket Policy

A bucket policy granting public `s3:GetObject` access to all objects in `cloud-gallery-nigatie` was added via the **Permissions** tab so that the static website's assets could be read anonymously by browsers:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::cloud-gallery-nigatie/*"
    }]}
```

Figure 9: Screenshot 8 – AWS CLI listing `cloud-gallery-nigatie` using the new IAM user's credentials.

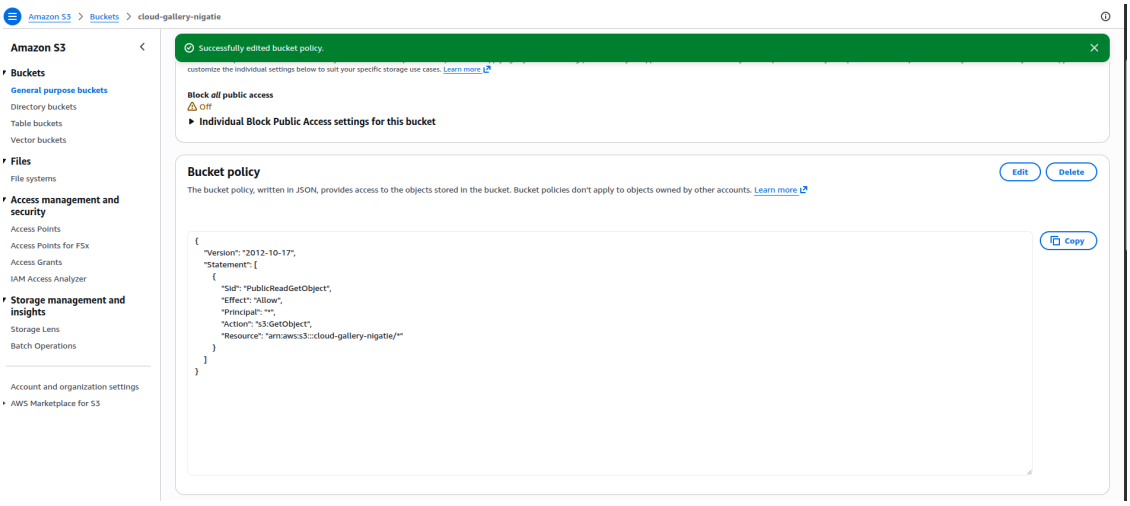

Figure 10: Screenshot 9 – Bucket policy successfully added, granting public read access to objects.

4.2 Task 4.2: Enable S3 Versioning

Bucket versioning was enabled from the **Properties** tab so that every version of an uploaded object is preserved.

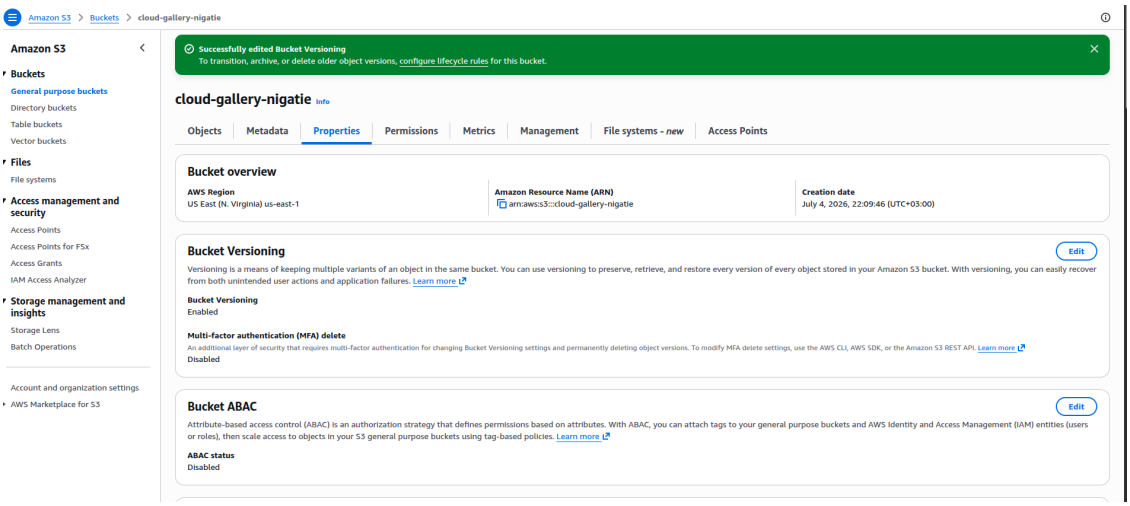

Figure 11: Screenshot 10 – Bucket Versioning set to "Enabled" for `cloud-gallery-nigatie`.

4.3 Task 4.3: S3 Lifecycle Rule

A lifecycle rule named `move-to-glacier` was created for the entire bucket, transitioning current object versions to a lower-cost storage class (Glacier Flexible Retrieval) after a set number of days.

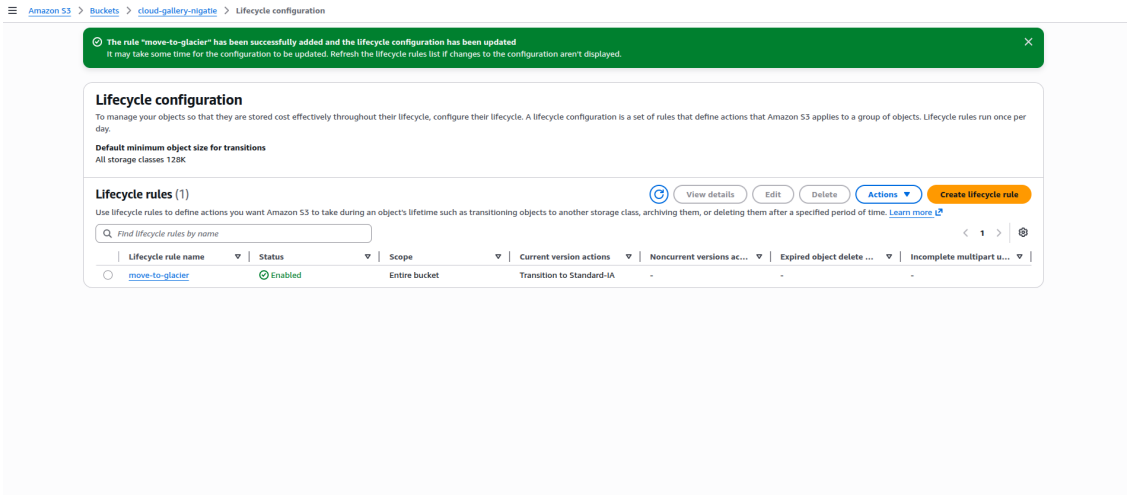

Figure 12: Screenshot 11 – Lifecycle rule "move-to-glacier" successfully created and enabled.

4.4 Task 4.4: CloudWatch Monitoring

Status: Not completed – AWS Free Tier account access issue.

After completing the lifecycle rule configuration in Task 4.3, my AWS Free Tier account stopped working and I was unable to sign back into the AWS Management Console to access CloudWatch. As a result, I was not able to capture the required CloudWatch metric graph screenshot, view `BucketSizeBytes` for the bucket, or create the billing/storage alarm described in the assignment.

5 Troubleshooting & Clean Up

Status: Not completed – AWS Free Tier account access issue.

Because the AWS account became inaccessible after the CloudWatch step, I was unable to complete the remaining hands-on tasks in this project:

- **Task 5.1 (Simulate and Fix an Issue):** renaming `photo1.jpg` to `photo1-hidden.jpg` via `aws s3 mv`, observing the broken image on the live site, checking CloudWatch Logs, and restoring the file.
- **Part 6 (Clean Up):** emptying and deleting the S3 bucket (`aws s3 rm --recursive` / `aws s3 rb`) and deleting the `s3-uploader` IAM user and its access key.

As a result, the `cloud-gallery-nigatie` bucket, its objects, and the `s3-uploader` IAM user and access key currently pending account access recovery, and no clean-up screenshots could be captured.

5.1 Task 5.2: Reflection Questions

1. **What is the difference between making an S3 object public and using a bucket policy?**
   Making an individual object public (via an ACL) grants read access to that single object only, and has to be repeated for every new file uploaded. A bucket policy, by contrast, is a single JSON document attached to the whole bucket that can grant (or restrict) access to every current and future object matching a resource pattern, which is why I used a bucket policy with `s3:GetObject` on `cloud-gallery-nigatie/*` instead of setting ACLs per file.

2. **Why is S3 versioning important for a website?**
   Versioning keeps every previous copy of an object instead of overwriting it, so if a website file such as `index.html` or a photo is accidentally deleted, corrupted, or replaced with a bad version, it can be restored from an earlier version rather than being lost permanently.

3. **How does CloudWatch help us monitor your S3 storage?**
   CloudWatch collects metrics such as `BucketSizeBytes` and object counts for a bucket over time, and lets us set alarms (for example, when storage exceeds 100 MB) that notify us by email through SNS, so that storage growth or unusual activity can be caught early instead of only being discovered on the monthly bill.

4. **What is the purpose of an S3 lifecycle rule?**
   A lifecycle rule automatically transitions or expires objects based on their age, moving infrequently accessed data such as older photos to cheaper storage classes like Standard-IA or Glacier, which reduces storage cost without requiring any manual intervention.

5. **Why would you create an IAM user for S3 access instead of using the root account?**
   An IAM user such as `s3-uploader` can be given only the specific permissions it needs (in this case, S3 access), and its credentials can be rotated or revoked independently, whereas the root account has unrestricted access to every service and billing setting in the account, so using it for routine tasks unnecessarily increases the damage a leaked credential could cause.

Part	Task	Points	Status
Part 1	Create S3 Bucket	10	Complete
	Upload Images	10	Complete
Part 2	Static Hosting Configuration	10	Complete
	Create Website Files	10	Complete
	Test Website	5	Complete
Part 3	Create IAM User	10	Complete
	Generate Access Keys	5	Complete
	Configure AWS CLI	5	Complete
Part 4	Bucket Policy	10	Complete
	Versioning	5	Complete
	Lifecycle Rule	5	Complete
	CloudWatch Monitoring	5	Not completed (account inaccessible)
Part 5	Troubleshooting	10	Not completed (account inaccessible)
	Reflection Questions	5	Complete
Bonus	Clean Up	—	Not completed (account inaccessible)